



GitHub Trending Top 10

每日热门开源项目深度解读

2026-10-06

今日榜单

- 1** **tester-army/e2e** TypeScript 3天
Next generation e2e testing framework for web and mobile apps.
- 2** **mattpocock/skills** Shell NEW
Skills for Real Engineers. Straight from my .agents directory.
- 3** **earthtojake/text-to-cad** Python 3天
Give your agent CAD superpowers.
- 4** **boykopovar/AnyPS5** C++ 2天
Tool for automatic PS5 executables porting to Linux and Windows
- 5** **pbakaus/impeccable** JavaScript NEW
The design language that makes your AI harness better at design.
- 6** **thedomack/claude-mem** TypeScript 2天
Persistent Context Across Sessions for Every Agent – Captures everything your ag...
- 7** **ayghri/i-have-adhd** Python NEW
A skill to stop your coding agent from burying the answer. ADHD-friendly output.
- 8** **morluto/rea** TypeScript NEW
Reverse engineer anything with agents, from app behavior down to native binaries...
- 9** **deepseek-ai/DeepGEMM** Cuda NEW
DeepGEMM: clean and efficient BLAS kernel library on GPU
- 10** **msitarzewski/agency-agents** Shell NEW
A complete AI agency at your fingertips - From frontend wizards to Reddit commun...

#1

TypeScript

连续 3 天

e2e

Next generation e2e testing framework for web and mobile apps.

Stars **5,888** Forks **263** Today **+1,720**

<https://github.com/tester-army/e2e>

项目简介：e2e 是 TesterArmy 团队推出的下一代端到端测试框架，支持 Web 和移动应用。它最大的特点是允许开发者用自然语言描述测试目标，由 AI Agent 自动驱动应用完成操作，从而大幅降低编写和维护 E2E 测试的门槛。

核心功能：

- **自然语言驱动测试：**通过 `agent.act()` 和 `agent.assert()` 用自然语言描述操作和断言，AI Agent 自动执行。
- **录制与回放机制：**Agent 执行过的步骤会被记录下来，下次运行时直接回放，无需调用模型，直到应用发生变化。
- **多端支持：**Web 端基于 Playwright 支持 Chromium/Firefox/WebKit，移动端支持 iOS 模拟器和 Android 模拟器。
- **灵活接入模型：**支持自带订阅、API Key 或本地模型，无 Agent 步骤的测试完全不需要模型。
- **CI 集成：**提供 GitHub Reporter，可将测试结果直接以 PR 评论形式呈现。

适用场景：适合需要维护大量 E2E 测试的前端和移动端团队，尤其是那些因 UI 频繁变动导致测试脚本维护成本高的项目。也适合希望让非技术成员（如产品、QA）参与编写测试的团队。

技术亮点：最值得关注的是"Agent 执行 + 确定性回放"的混合模式——首次运行依赖 AI 理解自然语言并执行，后续运行复用录制的动作序列，兼顾了灵活性和稳定性。此外，它把 Playwright、移动模拟器等底层能力封装成独立 npm 包（@e2e-dev/*），架构清晰且可组合。

上手建议：直接运行 `npx e2e init`，按提示选择引擎（Web/移动）和模型提供商即可生成配置和示例测试。想快速了解完整配置，可以查看仓库 `examples/` 目录下的 Vite、Next.js、Expo、SwiftUI 示例项目。贡献代码前建议先阅读 `CONTRIBUTING.md`，有问题可通过 Discord 社区交流。

#2

Shell

NEW

skills

Skills for Real Engineers. Straight from my .agents directory.

Stars **277,775** Forks **23,268** Today **+1,028**

<https://github.com/mattpocock/skills>

项目简介：这是 TypeScript 社区知名教育者 Matt Pocock 开源的 AI Agent 技能集（Skills），从他自己日常使用的 .agents 目录中提取而来。它要解决的核心问题是：像 GSD、BMAD、Spec-Kit 这类"全流程托管"方案虽然能帮 AI 编码，但会剥夺开发者对过程的控制权，一旦出错很难定位修复——而本项目主张用小而可组合的技能，让工程师保持主导权。

核心功能：一是"拷问式对齐"技能，如 /grill-me 和 /grill-with-docs，让 Agent 主动向你提问、澄清需求，从源头减少"做出来的不是我想要的"这类最常见失败；二是 /triage 等与 issue tracker（GitHub/GitLab/本地文件）和标签体系联动的分诊技能；三是 /setup-matt-pocock-skills 一次性配置脚本，按仓库设定 issue 跟踪方式、标签和文档存放位置；四是提供 Claude Code 插件与 skills.sh 两条安装路径，分别对应"订阅托管"和"复制可改"两种哲学。

适用场景：适合用 Claude Code、Codex 等编码 Agent 做**真实工程项目**（而非"vibe coding"快速原型）的开发者。典型场景是：需求模糊时先让 Agent 拷问你对齐目标、日常 issue 分诊与标签管理、以及希望把 Agent 工作流固化成自己可维护的一套规范时。

技术亮点：项目本身是 Shell 脚本 + Markdown 技能文件，轻量且模型无关（works with any model），强调"small, easy to adapt, composable"。设计上刻意与"大而全的流程框架"反着走——不接管你的流程，而是提供可拆卸的小积木。同时通过 ADR（架构决策记录）文档公开设计取舍，并把 Claude Code 官方市场插件与自建 marketplace 的更新延迟问题讲得很清楚，工程透明度高。

上手建议：最快路径是执行 `claude plugins install mattpocock-skills`（Claude Code 用户）或 `npx skills@latest add mattpocock/skills`（其他 Agent），记得勾选 `setup-matt-pocock-skills`，然后在仓库里跑一次 `/setup-matt-pocock-skills` 完成配置。想深入定制或贡献，建议从 `skills/` 目录下的单个 SKILL.md 读起，并参考 `.agents/adr/` 了解设计动机；作者还维护着约 6 万开发者的 newsletter，可跟进技能更新。

(注：README 中给出的总星标 277775 与今日新增 1028 的量级明显异常，疑似数据有误，建议以 GitHub 实际页面为准。)

#3

Python

连续 3 天

text-to-cad

Give your agent CAD superpowers.

Stars **17,806** Forks **1,795** Today **+620**

<https://github.com/earthtojake/text-to-cad>

项目简介：text-to-cad 是一个给 AI Agent 赋予"CAD 超能力"的插件/技能包，让 Claude Code、Cursor、Codex 等主流 Agent 能够通过自然语言直接生成 3D 模型文件。它解决的核心问题是：过去从想法到可制造的 CAD 模型需要专业软件和技能，现在只需对话即可完成建模、检查与出图。

核心功能： - 生成 STEP、GLB、STL、3MF 等多种格式的 3D 模型文件，覆盖从工程到 3D 打印的主流格式。 - 提供面向制造的设计检查（DFM）和工程图纸生成，不只是"能建模"，还关注"能造出来"。 - 对接 3D 打印、钣金、CNC 加工等制造服务，打通从设计到生产的链路。 - 内置本地查看器（cadgen mcp 本地服务器），可在对话中以可旋转的卡片形式预览模型。

适用场景：硬件工程师、产品设计师、创客和机械专业学生，在需要快速把想法变成可制造模型的场景下使用；也适合没有 CAD 软件使用经验的开发者，用自然语言完成简单零件设计并直接送厂加工。

技术亮点：底层基于 build123d（Python 参数化 CAD 库）和 Open CASCADE 7.9 几何内核，保证工业级建模精度；通过 MCP 协议和 skills 框架与各类 Agent 集成，兼容性广；用 uv 管理 Python 运行时，首次启动自动下载依赖，安装体验接近"一句话搞定"。

上手建议：最快的方式是直接对 Agent 说"Install text-to-cad from <https://github.com/earthtojake/text-to-cad>"，让它自动安装；手动安装需先确认 uv 已就绪。想贡献代码的话，注意 README 提到 main 分支是开发分支、latest 是稳定发布分支，建议从文档（texttocad.dev）和现有 skills 实现入手理解架构。

#4

C++

连续 2 天

AnyPS5

Tool for automatic PS5 executables porting to Linux and Windows

Stars **5,724** Forks **427** Today **+943**

<https://github.com/boykopovar/AnyPS5>

项目简介：AnyPS5 是一个旨在将 PS5 可执行文件自动移植到 Linux 和 Windows 平台的开源工具。它试图绕开传统模拟器路线，让 PS5 游戏以原生格式在 PC 上运行，从而解决跨平台兼容与游戏保存的问题。

核心功能：

- **重链接器 (Relinker)：**将 PS5 可执行文件转换为目标系统的原生格式，而非通过模拟层运行。
- **系统 PRX 库实现：**提供适用于动态链接的 PS5 系统库替代实现，支撑游戏运行所需的底层调用。
- **着色器重编译器：**将 PS5 着色器转换为 SPIR-V，并可通过 Spirv-Tools 进行验证。
- **输入映射支持：**兼容 SDL 手柄（含摇杆和扳机），并支持通过 ini 文件配置键鼠操作。

适用场景：主要面向游戏保存研究者、逆向工程爱好者以及希望在没有 PS5 硬件的情况下体验特定独占游戏的玩家。目前兼容性有限，已验证可运行的游戏如《Dreaming Sarah》可在中端 PC 上稳定 60 帧，适合用于技术验证和早期测试。

技术亮点：项目明确拒绝模拟器或独立运行时进程，而是通过重链接和原生库替换实现“原生移植”，这在技术路线上与 RPCS3 等模拟器形成鲜明对比。所有不支持的状态会严格抛出 `std::runtime_error` 并终止进程，体现了对未定义行为的零容忍设计哲学。

上手建议：建议先阅读 `docs/user/USAGE.md` 了解基本用法，并查阅 `COMPATIBILITY.md` 确认目标游戏是否在支持列表中。开发者可参考 `docs/dev/BUILD.md` 进行构建，并注意项目列出的技术债务文档，以了解当前实现的局限性。

#5

JavaScript

NEW

impeccable

The design language that makes your AI harness better at design.

Stars **77,500** Forks **4,617** Today **+609**

<https://github.com/pbakaus/impeccable>

项目简介：Impeccable 是一套面向 AI 编程助手（如 Claude）的“设计语言”技能包，旨在解决 AI 生成前端界面千篇一律、缺乏设计品味的问题。它通过结构化的命令和规则，引导 AI 产出更专业、更有辨识度的 UI 设计。

核心功能：一是 `/impeccable init` 一次性初始化，将产品定位、受众、约束等持久性事实写入 `PRODUCT.md`，避免后续命令重复沟通；二是提供 24 个设计命令（如 `polish`、`audit`、`critique`、`bolder`、`quieter`），构建人与 AI 共享的设计词汇表；三是内置 60 条确定性检测规则，CLI 和浏览器扩展无需 LLM 或 API Key 即可运行，用于自动识别 AI 生成设计的常见“套路”。

适用场景：主要面向使用 AI 编码工具（尤其是 Claude）进行前端开发的工程师和设计师，尤其适合需要快速产出高质量 UI 原型的场景。也适合希望统一团队设计语言、减少 AI 生成“模板感”的项目。

技术亮点：采用“技能 + 命令 + 确定性规则”三层架构，将设计指导从模糊的提示词转为可执行、可复用的工作流；确定性检测规则与 LLM 评审分离，前者零依赖、可离线运行，后者负责主观审美判断；还支持浏览器内实时迭代（live 模式）和元素变体生成，形成闭环。

上手建议：在项目根目录运行 `npx impeccable install`，然后在 AI 工具中执行 `/impeccable init` 完成初始化。想贡献可从新增检测规则或设计命令入手，也可参考官方文档和 Neo Mirai 案例研究理解实际效果。

#6

TypeScript

连续 2 天

claude-mem

Persistent Context Across Sessions for Every Agent – Captures everything your agent does during sessions, compresses it with AI, and injects relevant context back into future sessions. Works with Claude Code, OpenClaw, Codex, Gemini, Hermes, Copilot, OpenCode + More

Stars **96,986** Forks **8,548** Today **+536**

<https://github.com/thedotmack/claude-mem>

AI 分析生成失败: Error code: 400 - {'error': {'message': 'Content Exists Risk (request_id: 0ddab78a-eecc-4324-b93d-12eeff27be34)', 'type': 'invalid_request_error', 'param': None, 'code': 'invalid_request_error'}}

#7

Python

NEW

i-have-adhd

A skill to stop your coding agent from burying the answer. ADHD-friendly output.

Stars **54,229** Forks **3,116** Today **+318**

<https://github.com/ayghri/i-have-adhd>

项目简介：这是一个给 AI 编程助手（coding agent）用的"技能/插件"，核心目的是强制 AI 把答案放在最前面，而不是用一堆铺垫、寒暄和无关信息把真正的结论埋起来。它解决的是当前 AI 助手回复冗长、绕圈子、让人翻半天才找到关键操作的问题。

核心功能：一是"行动优先"，回复开头直接给出下一步该做什么，而不是先分析背景；二是多步骤任务强制编号，并限定列表不超过 5 项；三是每轮对话都重述当前状态、结尾只给一个具体的下一步，同时禁用"希望有帮助"这类无意义收尾语。此外还要求用具体分钟数（而非"一会儿"）给出时间估计，并把报错写得平实直白、不渲染情绪。

适用场景：主要面向使用 Claude 等支持 skill/plugin 机制的 AI 编程助手、且对冗长回复感到烦躁的开发者。典型场景是调试、改代码、跑测试这类需要快速拿到可执行指令的时刻，也适合自认为"注意力容易被打断"、希望信息密度高的使用者。

技术亮点：项目本体其实非常轻——核心逻辑集中在 SKILL.md 这份规则文档里，用 10 条明确规则约束 LLM 的输出风格，而非复杂的代码工程。它的巧妙之处在于把"成人 ADHD 应对工具"的思路迁移到 LLM 输出设计上，并提供了 fork 后自定义规则的路径，让用户能按自己偏好调整行为。

上手建议：最快的方式是直接把 README 里那段安装提示复制进 CLI，让助手按 AGENTS.md 自行安装；想深入定制的话，就 fork 仓库、修改 skills/i-have-adhd/SKILL.md 后替换上游版本。贡献方向上，由于项目本质是"提示词规则集"，最有价值的参与方式是提出更精炼的规则或补充多语言文档。

#8

TypeScript

NEW

rea

Reverse engineer anything with agents, from app behavior down to native binaries.

Stars **7,741** Forks **860** Today **+2,963**

<https://github.com/morluto/rea>

项目简介：REA 是一个基于 MCP（Model Context Protocol）的逆向工程工具集，让 AI Agent 能够直接分析原生二进制、JavaScript/Electron 应用、.NET 程序集和网站，从而理解某个功能的实现原理。它解决的核心问题是：当你看到一个应用中的功能想在自己的产品中复刻时，不必依赖源码，而是让 Agent 通过逆向分析给出有证据支撑的解释和实现方案。

核心功能：一是多目标逆向分析，覆盖原生二进制、Electron、.NET 和 Web 应用；二是 CLI 与 MCP 双入口，既可在终端直接使用，也可作为 Agent 工具调用；三是引导式安装，可自动配置 Agent 并接入已有的 Hopper/Ghidra；四是"调查模型"三阶段——反编译、理解、重建，把逆向结果直接转化为自己项目中的功能代码。

适用场景：适合想从成熟产品中学习功能实现方式的开发者、需要快速理解第三方闭源应用行为的安全研究人员，以及希望让 AI Agent 具备"看一眼就懂"能力的工程团队。典型用法是让 Agent 分析某个 App 的搜索功能，展示证据链，然后在自己技术栈中重建类似实现。

技术亮点：将逆向工程能力封装为标准 MCP 工具目录，让任意支持 MCP 的 Agent 都能调用；分析完全在本地运行，不上传目标应用，兼顾隐私与合规；结论附带证据和局限性说明，明确不声称能还原原始源码，态度务实；静态 JS 分析甚至不需要 Hopper/Ghidra 引擎。

上手建议：最快路径是运行 `npx rea-agents setup`，按引导完成 Agent 注册和工具接入，重启 Agent 后直接用自然语言提问。想贡献代码可先阅读 README 中的 Roadmap 和 How it works 部分，了解工具目录结构和 MCP 集成方式，再从 CI 流程和现有工具实现入手。

#9

Cuda

NEW

DeepGEMM

DeepGEMM: clean and efficient BLAS kernel library on GPU

Stars **8,612** Forks **1,365** Today **+363**

<https://github.com/deepseek-ai/DeepGEMM>

项目简介：DeepGEMM 是 DeepSeek 开源的一个面向 NVIDIA GPU 的高性能张量核心（Tensor Core）算子库，用统一、简洁的 CUDA 代码实现了大语言模型训练与推理中的核心计算原语。它解决的是「如何在保持代码可读性的同时，让 GEMM 等关键算子在各种矩阵形状下达到甚至超越专家手调库性能」这一问题。

核心功能： - 覆盖多种精度的 GEMM：FP8、FP4、BF16 的稠密与分组（Grouped）矩阵乘法。 - 融合 MoE 计算与通信重叠的 Mega MoE，以及面向 Lightning Indexer 的 MQA 打分、HyperConnection（HC）等 LLM 专用算子。 - 通过 DeepJIT 在运行时编译内核，安装时无需 CUDA 编译，降低部署门槛。

适用场景：主要面向做大模型训练/推理的工程团队与 GPU 内核开发者，尤其是在 H800、SM90/SM100 架构上榨取 FP8/FP4 算力、优化 MoE 与注意力打分性能的场景。

技术亮点：借鉴 CUTLASS/CuTe 思想但刻意摆脱重型模板依赖，只保留少量核心 kernel 函数，兼顾性能与可读性；H800 上实测可达约 1550 TFLOPS；SM100 支持全部内存布局（NT/TN/NN/TT），并针对 SM90/SM100 采用不同的缩放因子格式（FP32 与 UE8M0）。

上手建议：先确保满足 SM90/SM100 GPU、CUDA 12.9+、PyTorch 2.3+ 等依赖，用 `git clone --recursive` 拉取子模块后执行 `develop.sh` 或 `install.sh`，再从 `fp8_gemm_nt` 等接口入手跑通示例；想贡献可从 README 中列出的 PR 与内核优化方向切入。

#10

Shell

NEW

agency-agents

A complete AI agency at your fingertips - From frontend wizards to Reddit community ninjas, from whimsy injectors to reality checkers. Each agent is a specialized expert with personality, processes, and proven deliverables.

Stars **157,654** Forks **25,413** Today **+621**

<https://github.com/msitarzewski/agency-agents>

项目简介：这是一个精心策划的 AI Agent 角色库（Persona Collection），把不同职能的“AI 专家人格”以 Markdown 文件的形式沉淀下来——从前端工程师到 Reddit 社区运营、从创意注入者到现实检验者。它解决的核心问题是：大多数人在使用 Claude Code、Cursor 等 AI 编程工具时，只能得到通用化的回答，而缺乏“带着专业身份、流程和交付标准”的专家型输出。

核心功能：一是提供按职能分类的 Agent 人格库（工程、安全、设计、策略等），每个 Agent 都有独立人格、工作流和交付物示例；二是提供 `install.sh` / `convert.sh` 脚本，一键把 Agent 安装到 Claude Code、Cursor、Copilot、Gemini CLI、Aider 等十余种主流 AI 工具中；三是支持按团队或单个 Agent 精细安装（如 `--division engineering` 或 `--agent frontend-developer`）；四是提供原生桌面 App（macOS/Linux/Windows）实现可视化浏览与一键安装；五是 `runbooks.json` 让一组 Agent 能按预设阵容协同工作。

适用场景：适合重度使用 AI 编程助手（Claude Code、Cursor、Copilot 等）的开发者，希望通过“切换专家角色”让 AI 输出更贴近真实岗位的交付质量；也适合团队把内部最佳实践封装成标准化 Agent 模板；对内容创作者、社区运营者而言，Reddit/营销类 Agent 同样可用。简言之，任何想让 AI 从“通用助手”变成“专业同事”的人都是目标用户。

技术亮点：项目本身几乎全是 Markdown + Shell，没有任何运行时依赖，却通过 `convert.sh` 把同一份 Agent 定义转换成适配十多种工具的格式，这种“单一数据源、多目标编译”的思路非常优雅。把 AI 人格当作可版本管理、可分发、可组合的“软件资产”，而非一次性 Prompt，是这个项目最值得关注的理念创新。

上手建议：最省事的方式是直接下载桌面 App 或 `brew install --cask`；命令行用户可从 `./scripts/install.sh --list teams` 开始，先看清有哪些团队和 Agent，再用 `--division` 或

--agent 精确安装到自己的工具。想贡献的话，最直接的路径是新增或改进某个 Agent 的 Markdown 文件（人格、流程、交付物），提 PR 即可——门槛低，但要求内容足够专业和“有灵魂”。

数据来源: github.com/trending

报告日期: 2026-10-06

由 DeepSeek AI 提供智能分析 | 自动生成